

CORA: Extended Correctness Report

Supplementary artifact accompanying the VLDB submission

Tanisha Fonseca
Concordia University
Montreal, Quebec, CA
t_fonse@live.concordia.ca

Harunari Takata
Concordia University
Montreal, Quebec, CA
tkthdev@gmail.com

Gengrui Zhang
Concordia University
Montreal, Quebec, CA
gengrui.zhang@concordia.ca

PVLDB Reference Format:

Tanisha Fonseca, Harunari Takata, and Gengrui Zhang. CORA: Extended Correctness Report. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at <https://github.com/tanishaf28/woc/tree/main>.

This report contains the full statements, remarks, and proofs of every safety and liveness result for CORA. The main paper (§5) proves two lemmas directly, **Fast-Path Safety** and **Slow-Path Safety**, and folds several supporting arguments into their proofs as prose rather than separate lemmas. This report gives each of those supporting arguments as its own numbered lemma with a full proof:

- Lemma 1 (Quorum Intersection) underlies every quorum argument in §5 of the main paper and is stated there without proof.
- Lemmas 2, 6, and 7 (fixed-coordinator, fail-over, fencing) together correspond to the main paper’s Lemma 1 (Fast-Path Safety), whose proof sketches all three cases in one paragraph.
- Lemma 8 (Read Certification) corresponds to the quorum-read/ CATCHUP sentence inside that same proof.
- Lemmas 3 and 4 correspond to the paragraph preceding the main paper’s Lemma 2 (Slow-Path Safety).
- Lemma 9 (Cross-Path Exclusion) corresponds to the paragraph following Lemma 2, used in Theorem 1 (Safety).
- Lemmas 10, 11, 12, and 13 correspond to the main paper’s Lemma 3 (Fast-Path Liveness) and Lemma 4 (Slow-Path Liveness), each likewise compressed from several cases into one proof.

Section and equation references below (e.g. §4.3) refer to the corresponding sections of the main paper.

1 MODEL

We model the system as n replicas, each assigned a non-negative weight per weight domain. For the fast path, object O_i uses the weight vector $\mathbf{w}^{O_i}$ with threshold $wt_i = \frac{1}{2} \|\mathbf{w}^{O_i}\|_1$. For the slow path, all dependent transactions use the global weight vector $\mathbf{w}^g$ with threshold $wt_g = \frac{1}{2} \|\mathbf{w}^g\|_1$. A set of replicas Q forms a valid

quorum in a domain if the sum of its members’ weights in that domain is strictly greater than the domain threshold.

LEMMA 1 (QUORUM INTERSECTION). *Fix a weight vector ($\mathbf{w}$) with threshold (wt) equal to half the total weight. Any two valid quorums Q and Q' under the same sequence number s in that domain intersect.*

Remark 1.1. Lemma 1 requires the same weight *assignment*, not merely the same threshold or the same domain. REASSIGNWEIGHTS (§4.3) permutes which replica holds which weight value across rounds. This does not mean two quorums under different assignments are necessarily disjoint, they may still share a replica but the lemma alone does not *guarantee* it, and a proof that relies on such intersection must establish it some other way (e.g. by construction, as in Lemma 2, or by first restricting to a common assignment, as in Lemma 6). Every use of Lemma 1 below is checked against this requirement explicitly. Lemma 8’s certification is applied independently to each position using the certificate corresponding to the path on which that position was committed: fast-path positions are certified against $\mathbf{w}^{O_i}$, while slow-path positions are certified against $\mathbf{w}^g$.

PROOF. Let $W = \|\mathbf{w}\|_1$ be the total weight, so $wt = W/2$. Both quorums are formed under the same sequence number s and draw from the same replica set with the same weight vector (either an object weight vector $\mathbf{w}^{O_i}$ or the global weight vector $\mathbf{w}^g$). Since each quorum is valid, its weight is strictly greater than $W/2$. If Q and Q' were disjoint, their combined weight would strictly exceed $W/2 + W/2 = W$, contradicting that the total available weight is W . Hence $Q \cap Q' \neq \emptyset$. $\square$

2 SAFETY

LEMMA 2 (FAST-PATH SAFETY, FIXED COORDINATOR). *While the coordinator for object O_i does not change, no two correct replicas commit different independent transactions at the same sequence number $S[O_i]$ in the fast path.*

PROOF. A fast-path commit at sequence number $s = S[O_i]$ requires a coordinator to collect a valid quorum Q for O_i at s , i.e. $AccWeight > wt_i$. By §4.3, REASSIGNWEIGHTS always retains the coordinator’s own weight across successive rounds, re-ranking only its followers; so long as the coordinator does not change, it is therefore a member of every quorum it forms for O_i , regardless of how many reassignment rounds occur between them, and regardless of how the underlying multi-set of weight values has been permuted across the other replicas.

Suppose two correct replicas committed different transactions $tx \neq tx'$ at the same s and unchanged coordinator r . Each commit rests on a valid quorum, Q and Q' , both formed for O_i under the

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

same sequence number s , each containing r by the argument above, so $Q \cap Q' \supseteq \{r\} \neq \emptyset$ this holds directly from self-vote membership, without appeal to Lemma 1. By the monotonicity check (§4.4, RE-SPOND), r accepts at most one new proposal per sequence number for O_i : a second, differing proposal at s is rejected because it does not strictly exceed r 's local $s_r[O_i]$, and an exact retransmission of the same (s, tx) is re-acknowledged. Hence r can have accepted at most one of tx, tx' as a genuinely new proposal, contradicting that both were committed. Therefore $tx = tx'$. $\square$

Next, we discuss safety on the slow path. Since the slow path involves a single global leader, correctness under a stable leader is straightforward: the leader assigns each dependent transaction a global sequence number exactly once and processes its queue in order, so no two transactions are committed at the same position. Correctness across a leadership change requires more care, and we recall the leader election of Raft and Cabinet on which CORA builds.

Same as Cabinet, CORA uses Raft's leader election mechanism for the slow path *without* incorporating weighted voting into the election itself; this is what lets election safety survive REASSIGN-WEIGHTS permuting W^g across rounds, since a physical vote count over a fixed replica set does not depend on which weight label a replica currently holds.

LEMMA 3 (ELECTION LOG COMPLETENESS). *An elected slow-path leader always holds the most up-to-date slow-path log.*

PROOF. Partition the n replicas into G_1 , the $t_g + 1$ replicas that most recently acknowledged a slow-path commit, and G_2 , the remaining $n - t_g - 1$ replicas. Since a slow-path commit requires only $t_g + 1$ acknowledging replicas (§4.5), only G_1 is guaranteed to hold the most up-to-date log; replicas in G_2 may lag. By Raft's log-comparison rule, a replica votes for a candidate only if the candidate's log is at least as up to date as its own, so a lagging candidate can be voted for only by other lagging replicas, i.e. by at most $|G_2| = n - t_g - 1$ replicas. Since CORA's election quorum is the physical count $n - t_g$ (§4.5), a lagging candidate cannot reach quorum. Hence the elected leader is not lagging: it holds the most up-to-date log. $\square$

LEMMA 4 (ELECTION SAFETY). *At most one leader is elected in a given term.*

PROOF. Each replica casts at most one vote per term. Two disjoint vote quorums of size $n - t_g$ each would require $2(n - t_g) \leq n$, i.e. $t_g \geq n/2$; but $t_g \leq f = \lfloor (n - 1)/2 \rfloor < n/2$ by construction (§4.1), so any two vote quorums of size $n - t_g$ must overlap, and at most one candidate can collect $n - t_g$ votes in a given term. $\square$

LEMMA 5 (SLOW-PATH SAFETY). *In the slow path, no two correct replicas commit different transactions at the same global sequence number S^g .*

PROOF. Every dependent transaction is forwarded to the single global leader of the current term, which assigns it a global sequence number $s = S^g$ before broadcasting. A follower replica may speculatively execute a proposal on receipt, but such execution is provisional and is not a commit: only the leader declares

a commit, and only after a valid global quorum ($\sum \text{weight} > wt_g$) forms.

Under a stable leader, safety is immediate: the leader assigns each s exactly once and processes its queue in order, so it never commits two different transactions at the same s , and no replica accepts a second proposal at s by the monotonicity check.

Across a leadership change: by Lemma 4, at most one leader holds office in a given term. By Lemma 3, that leader holds the most up-to-date slow-path log, so it holds every entry committed before the election, including the entry committed at s . This argument is a physical vote count over a fixed replica set and does not depend on the weight assignment in effect at the time of either the earlier commit or the election, so it holds regardless of how many REASSIGNWEIGHTS rounds separate them. The new leader therefore, never assigns a different transaction to an already-committed s .

Consequently, a follower that provisionally executed some tx' at s under a failed leader, where tx' was never committed, is forced to reconcile with the new leader on sync: since the new leader does not hold tx' at s , the provisional value is reverted and replaced by the committed entry. No two correct replicas therefore retain different committed transactions at the same s , so $tx = tx'$. $\square$

LEMMA 6 (COORDINATOR FAIL-OVER SAFETY). *If the coordinator for object O_i changes from r to r' following r 's failure, and r' runs RECOVER (§4.3) before proposing any new transaction, then (i) the value r' adopts as the basis for its next proposal is backed by a valid quorum no staler than the last value committed under r , and (ii) r' does not propose a transaction that regresses past that value.*

PROOF. Let s^* be the sequence number of the most recently committed transaction on O_i under r . This commit was validated by a write quorum Q_w , formed under some weight generation g_k (§4.3). A weight generation for O_i only advances after that object's own coordinator commits successfully. So at the moment r' begins RECOVER, the generation that validated s^* is either the current generation among the surviving replicas, or the one immediately before it. Every replica retains one hop of weight history (§4.3), allowing r' to reconstruct a single generation subset from responders that participated in either of the two most recent rounds. If no such subset reaches WT_i , for example because too many responders are further behind, RECOVER falls back to Case (b) below, which is unconditionally safe.

Case (a): the read self-certifies. Suppose this reconstructed subset's combined weight under g_k exceeds WT_i . Then the subset is itself a valid quorum under g_k , by the ordinary commit rule (§4.4). Both this subset and Q_w are now validated under the same generation g_k , so Lemma 1 applies directly: the two quorums intersect at some correct replica ρ . Since $\rho \in Q_w$, ρ acknowledged s^* . By the monotonicity property behind Lemma 2, ρ 's local sequence token only ever increases, so ρ 's token is still at least s^* when r' reads it. The highest sequence number r' observes across its read responses, $s_{\max}$, is therefore at least s^* , and r' adopts $(s_{\max}, tx_{\max})$ directly.

Case (b): the read cannot self-certify. Suppose no consistent-generation subset of r' 's read responses reaches WT_i . Then r' cannot trust $s_{\max}$ on the strength of the read alone (§4.4, "Reads"). Instead, r' re-proposes $(s_{\max}, tx_{\max})$ through one fresh round of

PROPOSE. This round forms its own quorum, seeded by r' 's self-vote, and Lemma 2 already shows a single fixed coordinator's own quorums are safe. So this write-back round commits safely in isolation. As for staleness: if any correct replica still remembers s^* , case (a)'s argument already puts a floor of s^* under $s_{\max}$. So the write-back either re-commits a value that was already committed, harmlessly, or commits for the first time a value that had only been speculatively accepted, but never a value older than s^* .

Either way, r' sets its local sequence counter $s_{\text{coord}}[O_i]$ to $s_{\max}$ before accepting any new client transaction (§4.4, NEXTSEQ). So r' 's first new proposal carries sequence number $s_{\max} + 1$, which is at least $s^* + 1$. The monotonicity check behind Lemma 2 means every correct replica rejects a proposal that does not strictly exceed its own local sequence number. So no correct replica ever accepts a transaction from r' that regresses past the value committed under r . $\square$

LEMMA 7 (COORDINATOR FENCING). *Let r coordinate O_i at epoch e , and let r' be designated at epoch $e + 1$ by a ring reassignment. If r' completes RECOVER before issuing any proposal, then no proposal from r at any epoch $e' \leq e$ can subsequently be accepted by a valid quorum for O_i , even if r remains alive and was suspected incorrectly.*

PROOF. The final round of RECOVER yields a set Q_R that is a valid quorum for O_i under a single installed generation g^* . Each $\rho \in Q_R$ sets its local epoch to $e + 1$, i.e.,

$$e_\rho \leftarrow e + 1.$$

By the precondition, RECOVER does not advance the weight generation, so g^* remains installed.

Local epochs are monotone, and both ISNEW and ISRETRY require

$$M_{\phi}.e \geq e_r.$$

Therefore, every $\rho \in Q_R$ rejects any proposal from r with epoch $e' \leq e$. Let A denote the set of replicas that acknowledge such a proposal. Then

$$A \cap Q_R = \emptyset.$$

If r validates the proposal under g^* , then

$$A \cup \{r\} \subseteq \mathcal{R} \setminus Q_R.$$

Since Q_R is a valid quorum under g^* ,

$$w_{g^*}(Q_R) > WT_i.$$

Hence,

$$w_{g^*}(\mathcal{R} \setminus Q_R) < WT_i,$$

and therefore $A \cup \{r\}$ cannot form a valid quorum.

If r instead validates under some generation $g \neq g^*$, then g is not installed. Installation of g requires its adoption by a valid quorum under the preceding installed generation g^* . However, by the argument above, r cannot obtain such a quorum after Q_R has fenced epoch e . Thus, g cannot become installed, and r cannot subsequently validate a proposal under g .

Therefore, after r' completes RECOVER, no proposal from the old coordinator r at any epoch $e' \leq e$ can be accepted by a valid quorum for O_i . $\square$

LEMMA 8 (READ CERTIFICATION). *Any value $(s_{\max}, tx_{\max})$ returned by a quorum read on object O_i was committed by a valid weighted quorum for O_i at $s_{\max}$.*

PROOF. A quorum read collects responses until their combined weight exceeds wt_i , then checks whether the subset of responders reporting the highest sequence number $s_{\max}$, restricted to a single consistent weight generation, itself exceeds wt_i (§4.4).

If it does, that subset is a valid quorum for $s_{\max}$ under the ordinary commit rule, independent of whatever quorum first produced $s_{\max}$. The read returns $(s_{\max}, tx_{\max})$ only in this case, so the returned value is quorum-backed by construction.

If no such subset exists, the read does not return $(s_{\max}, tx_{\max})$ on the strength of the read alone. RECOVER instead re-proposes it through a fresh PROPOSE round, which forms its own quorum seeded by a self-vote (Lemma 2). Only once this round commits is the value treated as certified.

In either branch, the value CORA ultimately returns or treats as recovered is backed by a quorum-validated commit at the moment it is certified, never merely by the largest sequence token observed at a single replica. $\square$

Remark 2.1. The range read used by CATCHUP (§4.1, §4.4) applies this certification independently at each sequence number in the requested range; Lemma 8 therefore applies unchanged to every entry the range read returns.

LEMMA 9 (CROSS-PATH EXCLUSION). *For any independent object O_i , no fast-path and slow-path rounds that both include O_i in their locked object sets are concurrently active.*

PROOF. A new fast-path round on O_i can only be initiated by O_i 's ring-owner (§4.2). The ring-owner does not admit a new fast-path round while a RESERVE(O_i) is active. The leader sends RESERVE(O_i) at the start of its round, but O_i 's ring-owner grants RESERVED(O_i) only once any fast-path round it is currently coordinating on O_i has resolved, and the leader waits for this acknowledgment before broadcasting M_{SP} ; the ring-owner subsequently admits no new fast-path round on O_i until it receives RELEASE(O_i), sent only after the corresponding slow-path round resolves. Thus, once a slow-path round starts, no new fast-path round can start on O_i until that slow-path round completes. Conversely, the leader does not start a slow-path round on O_i until any preceding fast-path round on O_i has resolved. Finally, LOCKALL ensures that a replica cannot execute the fast- and slow-path critical sections for O_i concurrently. Therefore, the critical sections of any fast-path and slow-path rounds involving O_i cannot overlap, proving the lemma. $\square$

THEOREM 1 (SAFETY). *No two correct replicas commit different transactions at the same sequence number for the same object.*

PROOF. (Sketch) Each object is declared independent or dependent at deployment, and a transaction's type follows from the objects it accesses: a transaction on a single independent object is independent and runs on the fast path, while a transaction on a dependent object, or one spanning more than one object, is dependent and runs on the slow path. This classification partitions transactions: each transaction is exactly one type and commits on exactly one path. The two paths also use disjoint sequence-number spaces, fast-path commits for an independent object O_i are numbered in its per-object sequence $S[O_i]$, while slow-path commits are numbered in the global sequence S^g , so a fast-path and a slow-path commit can never share a sequence number.

The only point at which a transaction touches both paths is a FALLBACK reroute, and it does not create a cross-path commit. A fast-path commit is declared only when the coordinator's accumulated weight exceeds wt_i ; a reroute occurs only on timeout; i.e., when that threshold was not reached, so the rerouted transaction was never committed on the fast path, and any provisional follower state is reverted by FALLBACK before the transaction is committed once on the slow path. Hence every committed transaction commits on exactly one path.

Agreement on the fast path follows from Lemma 2 while a given object's coordinator is unchanged, and from Lemma 6 across each coordinator change; together these cover every fast-path commit regardless of how many failovers O_i has undergone. Agreement on the slow path follows from Lemma 5, which now covers both a stable leader and a leadership change. Because independent and dependent transactions are disjoint sets, no transaction is ever committed under two different types. What remains is disagreement induced through an object accessed by both paths: by Lemma 9 (Cross-Path Exclusion), no fast-path and slow-path round touching the same object O_i are ever concurrently active, so every commit touching O_i fast-path or slow-path is totally ordered by the real-time order of these mutually exclusive rounds, including the update to O_i 's shared version marker $V[O_i]$ (§4). Combined with per-path agreement, this establishes agreement for the whole system. $\square$

3 LIVENESS

CORA assumes a partially synchronous network. We further assume that at most t replicas fail in each weight domain, so that the correct replicas always retain enough weight to form a valid quorum. Concretely, by the geometric weight assignment inherited from Cabinet (§2), the top $t + 1$ weights in a domain exceed its threshold while the top t stay below it, so any $t + 1$ surviving replicas form a quorum and no set of t failures can prevent one.

LEMMA 10 (FAST-PATH LIVENESS). *Under the failure assumption, every fast-path transaction on an independent object either commits or falls back to the slow path in finite time.*

PROOF. The coordinator seeds the accumulator with its own object weight, broadcasts the proposal to all replicas, and waits on a bounded timer. If acknowledgments whose combined object weight exceeds wt_i arrive before the timer expires, the transaction commits. By the failure assumption the surviving replicas for O_i carry weight above wt_i , so after stabilization the threshold is reachable. If the timer expires first, the coordinator broadcasts a failure message and reroutes the transaction to the slow path (Lemma 12). Both outcomes occur within a bounded time, so every fast-path call terminates. When a coordinator change triggers RECOVER (Lemma 6), the read itself terminates in bounded time under the same partial-synchrony assumption. If the read cannot self-certify, RECOVER's write-back branch issues one further PROPOSE round, which is itself an ordinary fast-path call and so terminates by this same lemma. Either branch of RECOVER therefore completes in bounded time, so RECOVER delays but never prevents eventual commit. $\square$

LEMMA 11 (READ LIVENESS). *Under the failure assumption, every quorum read on an independent object either returns a value or signals the client to retry, and a retry eventually returns a value.*

PROOF. A read broadcasts a request and collects responses until it holds a quorum by weight, exactly as a write does; by the failure assumption this quorum is reachable in bounded time. If the responses self-certify (§4.4), the read returns immediately. Otherwise the read cannot yet trust the highest value it observed, and returns a bounded-retry signal rather than blocking indefinitely.

This can only happen in the window following an uncompleted fast-path round, and RECOVER (Lemma 10) resolves that window in bounded time regardless of whether any particular read is pending, since RECOVER runs as part of ordinary coordinator failover and does not wait on client reads. Once RECOVER completes, O_i has a value backed by a quorum under a single, consistent generation, so a client that retries after RECOVER's bounded completion time reads into the self-certifying case above. $\square$

LEMMA 12 (SLOW-PATH LIVENESS). *Under the failure assumption and an eventually stable leader, every dependent transaction eventually commits.*

PROOF. A dependent transaction is forwarded to the global leader, which processes its queue in arrival order. For each transaction the leader assigns a global sequence number, broadcasts it, and accumulates global weights until the total exceeds wt_g . By the failure assumption the correct replicas' aggregate global weight exceeds wt_g , so after stabilization the threshold is reached before the timer expires and the transaction commits. Because the queue is processed in arrival order, every enqueued transaction is eventually served. If the leader fails, a new leader is elected in finite time (Lemma 13) and resumes draining the queue, so progress continues. $\square$

LEMMA 13 (LEADER LIVENESS). *If the leader fails, a new leader is elected in finite time, after which slow-path progress resumes.*

PROOF. A leader failure is detected by timeout, upon which a correct replica becomes a candidate and requests votes. The leader election is described the same as Cabinet's election mechanism: a candidate is elected once it collects votes from $n - t_g$ replicas, a physical count rather than a weighted threshold (§4.5). By the failure assumption at most $t \leq t_g$ replicas fail, so at least $n - t_g$ correct replicas remain and can grant votes to a candidate holding the most up-to-date log, forming a winning quorum after stabilization. By Lemma 3, the elected leader holds every committed entry and resumes sequencing, so the slow path makes progress in finite time. $\square$

THEOREM 2 (LIVENESS). *Under the failure assumption and an eventually stable leader, every submitted transaction eventually commits, and every submitted read eventually returns a value.*

PROOF. An independent transaction either commits on the fast path or, on timeout, is rerouted to the slow path (Lemma 10); in the latter case it commits as a slow-path transaction by Lemma 12. A dependent transaction commits directly by Lemma 12. Leader failures are resolved by election in finite time (Lemma 13), after which the elected leader resumes draining the queue. Reads terminate or retry into termination by Lemma 11. Since every path terminates and fallback, recovery, and retry each add only bounded additional delay, every submitted transaction commits and every submitted read returns a value in finite time. $\square$